

JavaScript Intermediate — Property Descriptors & Object Control

Section 16 — Property Descriptors & Object Control

JavaScript-এ object property শুধু একটা `key: value` pair না।

প্রতিটি property-এর কিছু **behavioral rules** থাকে—যেমন:

- property-এর value change করা যাবে কি না
- property loop বা `Object.keys()` -এ দেখা যাবে কি না
- property delete/configuration করা যাবে কি না
- property কীভাবে read/write করবে

এই rules-গুলোকে বলা হয় **Property Descriptors**।

এই section-এ আমরা শিখব:

1. Property descriptors
2. `writable`
3. `enumerable`
4. `configurable`
5. `Object.defineProperty()`
6. `Object.freeze()`
7. `Object.seal()`
8. `Object.preventExtensions()`
9. Enumerability
10. Property iteration
11. Own vs inherited properties
12. `Object.keys()`
13. `Object.getOwnPropertyNames()`
14. `Object.getOwnPropertySymbols()`
15. `Reflect.ownKeys()`
16. সবগুলোর মধ্যে relationship

1. Property Descriptor কী?

সাধারণভাবে আমরা object এমনভাবে লিখি:

```
const user = {  
  name: "Duke",  
}
```

```
    age: 31
  };
```

আমরা দেখি:

```
name → "Duke"
age  → 31
```

কিন্তু JavaScript internally প্রতিটি property-এর সাথে কিছু metadata রাখে।

যেমন:

```
name
├─ value
├─ writable
├─ enumerable
└─ configurable
```

এই metadata-কে বলা হয়:

Property Descriptor

2. Basic Property Descriptor

একটি normal object property-এর data descriptor সাধারণত এই চারটি গুরুত্বপূর্ণ attribute নিয়ে কাজ করে:

Descriptor	Meaning
value	Property-এর actual value
writable	Value পরিবর্তন করা যাবে কি না
enumerable	Enumeration-এ property দেখা যাবে কি না
configurable	Property descriptor/delete পরিবর্তন করা যাবে কি না

Example:

```
const user = {
  name: "Duke"
};
```

```
console.log(
  Object.getOwnPropertyDescriptor(user, "name")
);
```

সাধারণত output হবে এরকম:

```
{
  value: "Duke",
  writable: true,
  enumerable: true,
  configurable: true
}
```

3. Object.getOwnPropertyDescriptor()

কোনো object-এর একটি **own property**-এর descriptor দেখতে:

```
Object.getOwnPropertyDescriptor(object, property)
```

Example:

```
const user = {
  name: "Duke"
};

const descriptor =
  Object.getOwnPropertyDescriptor(user, "name");

console.log(descriptor);
```

Output:

```
{
  value: "Duke",
  writable: true,
  enumerable: true,
  configurable: true
}
```

এটা debugging এবং advanced object manipulation-এর জন্য খুব useful ।

4. value

`value` property-এর actual value ।

```
const user = {  
  name: "Duke"  
};
```

Descriptor:

```
console.log(  
  Object.getOwnPropertyDescriptor(user, "name").value  
);
```

Output:

```
Duke
```

5. writable

`writable` নির্ধারণ করে property-এর value assignment-এর মাধ্যমে পরিবর্তন করা যাবে কি না ।

```
const user = {  
  name: "Duke"  
};
```

Default:

```
writable: true
```

তাই:

```
user.name = "Shariar";  
  
console.log(user.name);
```

Output:

Shariar

6. writable: false

এখন property-কে non-writable করা যাক:

```
const user = {};  
  
Object.defineProperty(user, "name", {  
  value: "Duke",  
  writable: false  
});
```

এখন:

```
user.name = "Shariar";  
  
console.log(user.name);
```

Strict mode ছাড়া সাধারণত assignment silently fail করতে পারে।

Value থাকবে:

Duke

Strict mode-এ assignment error throw করবে।

7. writable Mental Model

```
writable: true  
  ↓  
value change করা যায়  
  
writable: false
```

↓
value change করা যায় না

Important:

`writable: false` মানে property delete করা যাবে না—এমন নয়।

Delete-এর জন্য `configurable` relevant।

8. `enumerable`

`enumerable` নির্ধারণ করে property সাধারণ enumeration-এর সময় দেখা যাবে কি না।

যেমন:

```
const user = {  
  name: "Duke",  
  age: 31  
};  
  
console.log(Object.keys(user));
```

Output:

```
["name", "age"]
```

দুইটি property-ই enumerable।

9. `enumerable: false`

```
const user = {};  
  
Object.defineProperty(user, "secret", {  
  value: "12345",  
  enumerable: false  
});  
  
console.log(Object.keys(user));
```

Output:

```
[ ]
```

কিন্তু property আসলে object-এ আছে:

```
console.log(user.secret);
```

Output:

```
12345
```

অর্থাৎ:

Non-enumerable property hidden না; শুধু সাধারণ enumeration থেকে বাদ পড়ে।

10. enumerable মানে **Private** নয়

এটা খুব important।

```
Object.defineProperty(user, "secret", {  
  value: "12345",  
  enumerable: false  
});
```

এতে `secret` private হয়ে যায় না।

এখনও:

```
console.log(user.secret);
```

কাজ করবে।

এমনকি:

```
console.log(
  Object.getOwnPropertyNames(user)
);
```

দিয়ে property পাওয়া যাবে।

11. configurable

`configurable` property-এর configuration control করে।

এটা নির্ধারণ করে property:

- delete করা যাবে কি না
- descriptor-এর কিছু attributes পরিবর্তন করা যাবে কি না
- property reconfigure করা যাবে কি না

Example:

```
const user = {
  name: "Duke"
};

console.log(
  Object.getOwnPropertyDescriptor(user, "name")
);
```

Normal object literal property-তে:

```
configurable: true
```

12. configurable: false

```
const user = {};

Object.defineProperty(user, "name", {
  value: "Duke",
  configurable: false
});
```

এখন:

```
delete user.name;
```

কাজ করবে না।

Strict mode-এ error হবে।

13. configurable খুব Important

একবার:

```
configurable: false
```

করা হলে property-কে আবার সাধারণভাবে configurable করা যায় না।

অর্থাৎ:

```
configurable: true
  ↓
configurable: false
  ↓
ফিরিয়ে true করা যায় না
```

এটা one-way restriction-এর মতো।

14. writable, enumerable, configurable একসাথে

```
const user = {};

Object.defineProperty(user, "name", {
  value: "Duke",
  writable: false,
  enumerable: false,
  configurable: false
});
```

এখন:

Value change?

```
user.name = "Rahim";
```

✗ না।

`Object.keys()` ?

```
Object.keys(user);
```

✗ `name` দেখাবে না।

Delete?

```
delete user.name;
```

✗ না।

কিন্তু:

```
user.name
```

এখনও:

```
Duke
```

15. `Object.defineProperty()`

Syntax:

```
Object.defineProperty(  
  object,  
  property,  
  descriptor  
);
```

Example:

```
const user = {};  
  
Object.defineProperty(user, "name", {  
  value: "Duke",  
  writable: true,  
  enumerable: true,  
  configurable: true  
});
```

এতে property তৈরি হবে।

16. `defineProperty()` দিয়ে Existing Property Modify

শুধু নতুন property তৈরি না, existing property-এর descriptor-ও modify করা যায়।

```
const user = {  
  name: "Duke"  
};  
  
Object.defineProperty(user, "name", {  
  writable: false  
});
```

এখন:

```
user.name = "Rahim";
```

value change হবে না।

17. Descriptor-এর Default Values

এটা খুব important।

Object literal:

```
const user = {  
  name: "Duke"  
};
```

এখানে property সাধারণত:

```
writable: true  
enumerable: true  
configurable: true
```

কিন্তু `Object.defineProperty()` ব্যবহার করলে unspecified descriptor values-এর default behavior ভিন্ন।

Example:

```
const user = {};  
  
Object.defineProperty(user, "name", {  
  value: "Duke"  
});
```

এক্ষেত্রে descriptor attributes unspecified থাকলে default হিসেবে `false` হয়।

অর্থাৎ effectively:

```
{  
  value: "Duke",  
  writable: false,  
  enumerable: false,  
  configurable: false  
}
```

এই কারণে `defineProperty()` ব্যবহারের সময় descriptor explicitly লেখা ভালো।

18. Data Descriptor

যে descriptor-এ:

```
value  
writable
```

থাকে, সেটাকে **Data Descriptor** বলা হয়।

Example:

```
Object.defineProperty(user, "name", {  
  value: "Duke",  
  writable: true,  
  enumerable: true,  
  configurable: true  
});
```

19. Accessor Descriptor

Accessor descriptor-এ `get` এবং `set` ব্যবহার করা হয়।

Example:

```
const user = {  
  firstName: "Shariar",  
  lastName: "Duke"  
};  
  
Object.defineProperty(user, "fullName", {  
  get() {  
    return `${this.firstName} ${this.lastName}`;  
  }  
});
```

এখন:

```
console.log(user.fullName);
```

Output:

```
Shariar Duke
```

`fullName` stored value নয়।

Getter dynamically value তৈরি করছে।

20. Getter + Setter with `defineProperty`

```
const user = {
  _name: "Duke"
};

Object.defineProperty(user, "name", {
  get() {
    return this._name;
  },
  set(value) {
    this._name = value.trim();
  }
});
```

Use:

```
user.name = " Sharar ";
console.log(user.name);
```

Output:

```
Sharar
```

21. Data Descriptor vs Accessor Descriptor

Data Descriptor	Accessor Descriptor
<code>value</code>	<code>get</code>
<code>writable</code>	<code>set</code>
<code>enumerable</code>	<code>enumerable</code>

Data Descriptor	Accessor Descriptor
configurable	configurable

একই property descriptor-এ সাধারণভাবে `value/writable` এবং `get/set` একসাথে ব্যবহার করা উচিত নয়।

22. `Object.getOwnPropertyDescriptors()`

একটি object-এর সব own property-এর descriptors একসাথে দেখতে:

```
Object.getOwnPropertyDescriptors(object)
```

Example:

```
const user = {
  name: "Duke",
  age: 31
};

console.log(
  Object.getOwnPropertyDescriptors(user)
);
```

এতে প্রতিটি property-এর descriptor পাওয়া যাবে।

23. কেন `getOwnPropertyDescriptors()` Useful?

Object-এর complete property definitions inspect করতে useful।

আর advanced copying-এর সময়ও useful:

```
const copy = Object.create(
  Object.getPrototypeOf(original),
  Object.getOwnPropertyDescriptors(original)
);
```

এটা শুধু values copy করার চেয়ে property descriptors preserve করতে পারে।

24. Object.freeze()

`Object.freeze()` object-কে strongly restricted state-এ নিয়ে যায়।

```
const user = {  
  name: "Duke",  
  age: 31  
};  
  
Object.freeze(user);
```

তারপর সাধারণভাবে:

- existing property value change করা যাবে না
- নতুন property add করা যাবে না
- property delete করা যাবে না
- property configuration change করা যাবে না

25. Freeze Example

```
const user = {  
  name: "Duke"  
};  
  
Object.freeze(user);  
  
user.name = "Shariar";  
user.age = 31;  
  
delete user.name;  
  
console.log(user);
```

Object পরিবর্তিত হবে না।

Strict mode-এ invalid mutation attempt error throw করতে পারে।

26. Object.isFrozen()

কোনো object frozen কিনা check করতে:

```
Object.isFrozen(object)
```

Example:

```
const user = {  
  name: "Duke"  
};  
  
Object.freeze(user);  
  
console.log(Object.isFrozen(user));
```

Output:

```
true
```

27. Object.seal()

`Object.seal()` object-এ নতুন property add এবং existing property delete/configure করা বন্ধ করে।

কিন্তু existing writable properties-এর value change করা যায়।

Example:

```
const user = {  
  name: "Duke",  
  age: 31  
};  
  
Object.seal(user);  
  
user.name = "Shariar";  
  
console.log(user.name);
```

Output:

```
Shariar
```

কিন্তু:

```
delete user.age;
```

✗ allowed নয়।

আর:

```
user.email = "test@example.com";
```

✗ নতুন property add করা যাবে না।

28. Object.isSealed()

```
const user = {  
  name: "Duke"  
};  
  
Object.seal(user);  
  
console.log(Object.isSealed(user));
```

Output:

```
true
```

29. Object.preventExtensions()

এটা তিনটির মধ্যে সবচেয়ে কম restrictive।

এটা শুধু নতুন property add করা বন্ধ করে।

```
const user = {  
  name: "Duke"  
};
```

```
Object.preventExtensions(user);
```

এখন:

```
user.age = 31;
```

✗ নতুন property add হবে না।

কিন্তু existing property:

```
user.name = "Shariar";
```

সাধারণভাবে পরিবর্তন করা যাবে।

এবং:

```
delete user.name;
```

সাধারণভাবে করা যাবে, যদি property configurable থাকে।

30. Object.isExtensible()

Object নতুন property accept করতে পারবে কি না check:

```
Object.isExtensible(object)
```

Example:

```
const user = {};  
  
console.log(Object.isExtensible(user));
```

Output:

```
true
```

তারপর:

```
Object.preventExtensions(user);  
  
console.log(Object.isExtensible(user));
```

Output:

```
false
```

31. Freeze vs Seal vs PreventExtensions

সবচেয়ে important comparison:

Feature	Prevent Extensions	Seal	Freeze
New properties	✗	✗	✗
Delete properties	✓*	✗	✗
Change existing values	✓*	✓*	✗
Reconfigure properties	কিছু ক্ষেত্রে	✗	✗

* property descriptor-এর ওপর নির্ভর করে।

সহজভাবে মনে রাখো:

```
preventExtensions  
  ↓  
শুধু নতুন property বন্ধ  
  
seal  
  ↓  
নতুন property + delete + reconfigure বন্ধ  
  
freeze  
  ↓  
seal-এর সব restriction  
+  
writable properties-ও non-writable
```

32. Restriction Levels

```
Normal Object
  ↓
preventExtensions()
  ↓
seal()
  ↓
freeze()
```

প্রতিটি ধাপে restriction বাড়ে।

33. Freeze কি Deep Freeze?

না।

এটা খুব important।

`Object.freeze()` **shallow**।

Example:

```
const user = {
  name: "Duke",
  address: {
    city: "Dhaka"
  }
};

Object.freeze(user);
```

এখন:

```
user.name = "Rahim";
```

❌ পরিবর্তন হবে না।

কিন্তু:

```
user.address.city = "Rangpur";
```

nested object frozen না থাকলে:

✓ পরিবর্তন হতে পারে।

34. কেন Freeze Shallow?

কারণ:

```
user
├── name → primitive
└── address → object reference
                ↓
            nested object
```

`Object.freeze(user)` outer object-কে freeze করে।

Nested object আলাদা object।

তাই:

```
Object.freeze(user.address);
```

না করলে nested object mutable থাকতে পারে।

35. Deep Freeze Concept

Recursive deep freeze করা সম্ভব।

Basic idea:

```
function deepFreeze(object) {
  Object.freeze(object);

  for (const value of Object.values(object)) {
```

```

    if (
      value !== null &&
      typeof value === "object" &&
      !Object.isFrozen(value)
    ) {
      deepFreeze(value);
    }
  }

  return object;
}

```

Use:

```

const user = {
  name: "Duke",
  address: {
    city: "Dhaka"
  }
};

deepFreeze(user);

```

এখন nested object-ও freeze হবে।

Production code-এ deep-freeze ব্যবহার করার আগে performance এবং actual requirement consider করা উচিত।

36. Object.freeze() কি Primitive-এ কাজ করে?

Modern JavaScript-এ Object.freeze() non-object values-এর ক্ষেত্রে meaningful object freezing করে না; primitives-এর immutability-এর concept আলাদা।

Primitives যেমন:

```

const name = "Duke";

```

নিজেই immutable value।

Object freezing মূলত object-এর property structure/control-এর জন্য।

37. Enumerability কী?

Enumeration মানে object-এর properties একে একে discover/iterate করা।

সব property enumeration-এ automatically আসে না।

এখানে:

```
enumerable: true
```

হলে property সাধারণ enumerable operations-এ দেখা যায়।

38. Object.keys()

শুধু:

- own
- enumerable
- string-keyed

properties দেয়।

Example:

```
const user = {  
  name: "Duke",  
  age: 31  
};  
  
console.log(Object.keys(user));
```

Output:

```
["name", "age"]
```

39. Non-Enumerable Property + `Object.keys()`

```
const user = {  
  name: "Duke"  
};  
  
Object.defineProperty(user, "secret", {  
  value: 123,  
  enumerable: false  
});  
  
console.log(Object.keys(user));
```

Output:

```
["name"]
```

`secret` বাদ যাবে।

40. `Object.values()`

`Object.values()` own enumerable string-keyed properties-এর values দেয়।

```
const user = {  
  name: "Duke",  
  age: 31  
};  
  
console.log(Object.values(user));
```

Output:

```
["Duke", 31]
```

41. `Object.entries()`

Key-value pairs দেয়:

```
const user = {
  name: "Duke",
  age: 31
};

console.log(Object.entries(user));
```

Output:

```
[
  ["name", "Duke"],
  ["age", 31]
]
```

Useful for:

```
for (const [key, value] of Object.entries(user)) {
  console.log(key, value);
}
```

42. for...in

`for...in` object-এর **enumerable string-keyed properties** iterate করে।

কিন্তু গুরুত্বপূর্ণ difference:

`for...in` own properties-এর পাশাপাশি inherited enumerable properties-ও দেখতে পারে।

Example:

```
const parent = {
  role: "admin"
};

const user = Object.create(parent);

user.name = "Duke";

for (const key in user) {
```

```
    console.log(key);  
  }
```

Potential output:

```
name  
role
```

role inherited।

43. `for...in`-এ Own Property Check

যদি শুধু own properties চাই:

```
for (const key in user) {  
  if (Object.hasOwn(user, key)) {  
    console.log(key);  
  }  
}
```

এতে inherited properties বাদ যাবে।

44. `Object.getOwnPropertyNames()`

এটা object-এর:

- own
- string-keyed

property names দেয়, **enumerable** হোক বা না হোক।

Example:

```
const user = {  
  name: "Duke"  
};  
  
Object.defineProperty(user, "secret", {  
  value: 123,
```

```
    enumerable: false
  });

console.log(Object.getOwnPropertyNames(user));
```

Output:

```
["name", "secret"]
```

45. Object.keys() vs getOwnPropertyNames()

Method	Own	Enumerable	String Keys
Object.keys()	✓	শুধু enumerable	✓
Object.getOwnPropertyNames()	✓	enumerable + non-enumerable	✓

46. Symbol Properties

JavaScript object-এর key শুধু string না।

Symbol key-ও হতে পারে।

```
const id = Symbol("id");

const user = {
  name: "Duke",
  [id]: 123
};
```

এখানে:

```
"name"
```

একটি string key।

আর:

```
id
```

একটি Symbol key।

47. `Object.getOwnPropertySymbols()`

Object-এর own Symbol keys পেতে:

```
Object.getOwnPropertySymbols(object)
```

Example:

```
const id = Symbol("id");

const user = {
  name: "Duke",
  [id]: 123
};

console.log(Object.getOwnPropertySymbols(user));
```

Output-এ Symbol পাওয়া যাবে।

48. `Object.keys()` Symbol Key দেখায় না

```
const id = Symbol("id");

const user = {
  name: "Duke",
  [id]: 123
};

console.log(Object.keys(user));
```

Output:

```
[ "name" ]
```

Symbol key এখানে থাকবে না।

49. Reflect.ownKeys()

যদি object-এর সব own keys একসাথে চাই:

```
Reflect.ownKeys(object)
```

এটা দেয়:

- enumerable string keys
- non-enumerable string keys
- Symbol keys

Example:

```
const id = Symbol("id");

const user = {
  name: "Duke",
  [id]: 123
};

Object.defineProperty(user, "secret", {
  value: "hidden",
  enumerable: false
});

console.log(Reflect.ownKeys(user));
```

এখানে `name`, `secret`, এবং Symbol key—সব পাওয়া যাবে।

50. Property Iteration Complete Comparison

এটা ভালোভাবে মনে রাখো:

Method	Own	Inherited	Non-enumerable	Symbols
<code>Object.keys()</code>	✓	✗	✗	✗
<code>Object.values()</code>	✓	✗	✗	✗
<code>Object.entries()</code>	✓	✗	✗	✗
<code>for...in</code>	✓	✓	✗	✗
<code>Object.getOwnPropertyNames()</code>	✓	✗	✓	✗
<code>Object.getOwnPropertySymbols()</code>	✓	✗	✓	✓
<code>Reflect.ownKeys()</code>	✓	✗	✓	✓

এই table interview এবং practical development—দুই জায়গাতেই useful।

51. `Object.getOwnPropertyNames()` vs `Reflect.ownKeys()`

ধরো:

```
const symbolKey = Symbol("id");

const user = {
  name: "Duke",
  [symbolKey]: 123
};

Object.defineProperty(user, "secret", {
  value: true,
  enumerable: false
});
```

তাহলে:

```
Object.getOwnPropertyNames(user);
```

পাবে:

```
["name", "secret"]
```

কিন্তু:

```
Reflect.ownKeys(user);
```

পাবে:

```
["name", "secret", Symbol(id)]
```

52. Object.entries() + for...of

Object iterate করার খুব common modern pattern:

```
const user = {  
  name: "Duke",  
  age: 31  
};  
  
for (const [key, value] of Object.entries(user)) {  
  console.log(key, value);  
}
```

Output:

```
name Duke  
age 31
```

এটা inherited properties নিয়ে `for...in` -এর মতো ambiguity তৈরি করে না, কারণ `Object.entries()` own enumerable properties দেয়।

53. Property Descriptor + Enumeration

এই দুই concept একসাথে বুঝা:

```
const user = {};  
  
Object.defineProperty(user, "visible", {  
  value: true,
```

```
    enumerable: true
  });

Object.defineProperty(user, "hidden", {
  value: false,
  enumerable: false
});
```

এখন:

```
Object.keys(user);
```

→

```
["visible"]
```

কিন্তু:

```
Object.getOwnPropertyNames(user);
```

→

```
["visible", "hidden"]
```

অর্থাৎ `enumerable` property iteration-এর behavior control করে।

54. Freeze-এর সাথে Descriptors

`Object.freeze()` internally object-এর properties-কে highly restricted করে।

Conceptually existing own properties:

```
writable → false
configurable → false
```

এবং object নতুন property accept করে না।

তাই:

```
const user = {
  name: "Duke"
};

Object.freeze(user);

console.log(
  Object.getOwnPropertyDescriptor(user, "name")
);
```

Descriptor-এ `writable` এবং `configurable` false থাকবে।

`enumerable` সাধারণত আগের মতোই থাকে।

55. Seal-এর সাথে Descriptors

`Object.seal()`:

`configurable` → `false`

করে এবং object-কে non-extensible করে।

কিন্তু writable properties সাধারণত writable থাকে।

তাই:

```
const user = {
  name: "Duke"
};

Object.seal(user);

user.name = "Shariar";
```

এখনও কাজ করবে।

56. Prevent Extensions-এর সাথে Descriptors

`Object.preventExtensions()`:

Object extensible → false

করে।

কিন্তু existing property-এর descriptor automatically non-writable বা non-configurable করে না।

তাই existing property-এর value change করা সম্ভব হতে পারে।

57. Three Object Control Methods — Mental Model

```
preventExtensions()  
  ↓  
"নতুন property add করো না"  
  
seal()  
  ↓  
"নতুন property add করো না  
existing property delete/reconfigure করো না"  
  
freeze()  
  ↓  
"নতুন property add করো না  
delete করো না  
reconfigure করো না  
value change করো না"
```

58. Object.isExtensible()

```
const user = {};  
  
console.log(Object.isExtensible(user));
```

→ true

তারপর:

```
Object.preventExtensions(user);  
  
console.log(Object.isExtensible(user));
```

→ false

59. Object.isSealed()

```
const user = {};  
  
Object.seal(user);  
  
console.log(Object.isSealed(user));
```

→ true

60. Object.isFrozen()

```
const user = {};  
  
Object.freeze(user);  
  
console.log(Object.isFrozen(user));
```

→ true

61. One Important Connection — const vs freeze

এগুলো এক জিনিস না।

```
const user = {  
  name: "Duke"  
};
```

```
user.name = "Shariar";
```

এটা allowed।

কারণ `const` variable binding change করতে দেয় না:

```
user = {};
```



কিন্তু object-এর property change:

```
user.name = "Shariar";
```



অন্যদিকে:

```
Object.freeze(user);
```

object properties-ও restrict করে।

মনে রাখো:

```
const
↓
variable binding protect

freeze
↓
object properties restrict
```

62. Shallow Copy + Descriptors

Spread:

```
const copy = {  
  ...user  
};
```

সাধারণত property values copy করে, কিন্তু original property descriptors-এর সব details preserve করে না।

যদি descriptor preservation দরকার হয়:

```
const copy = Object.create(  
  Object.getPrototypeOf(user),  
  Object.getOwnPropertyDescriptors(user)  
);
```

এটা advanced object cloning technique।

63. Common Mistakes

Mistake 1 — `enumerable: false` মানে **private** ভাবা

ভুল।

```
enumerable: false
```

শুধু সাধারণ enumeration থেকে property বাদ দেয়।

Mistake 2 — `writable: false` মানে **delete** করা যাবে না ভাবা

ভুল।

Delete control মূলত:

```
configurable
```

এর সাথে related।

Mistake 3 — `Object.freeze()` **deep** ভাবা

ভুল।

`freeze()` shallow।

Mistake 4 — `seal()` **value change** বন্ধ করে

ভুল।

`seal()` existing writable properties-এর value change allow করতে পারে।

Mistake 5 — `preventExtensions()` সব **modification** বন্ধ করে

ভুল।

এটা মূলত নতুন properties add করা বন্ধ করে।

Mistake 6 — `Object.keys()` সব **properties** দেয়

ভুল।

এটা শুধু:

own + enumerable + string-keyed

properties দেয়।

Mistake 7 — `for...in` শুধু **own properties** দেয়

ভুল।

Inherited enumerable properties-ও আসতে পারে।

64. Output Prediction — Example 1

```
const user = {};  
  
Object.defineProperty(user, "name", {  
  value: "Duke",  
  writable: false  
});  
  
user.name = "Rahim";  
  
console.log(user.name);
```

Output:

Duke

কারণ `writable: false` ।

65. Output Prediction — Example 2

```
const user = {  
  name: "Duke"  
};  
  
Object.defineProperty(user, "secret", {  
  value: 123,  
  enumerable: false  
});  
  
console.log(Object.keys(user));
```

Output:

["name"]

66. Output Prediction — Example 3

```
const user = {  
  name: "Duke"  
};  
  
Object.seal(user);  
  
user.name = "Rahim";  
user.age = 31;  
  
console.log(user.name);  
console.log(user.age);
```

Output conceptually:

```
Rahim  
undefined
```

কারণ existing `name` writable ছিল, কিন্তু নতুন `age` add করা যায়নি।

67. Output Prediction — Example 4

```
const user = {  
  name: "Duke"  
};  
  
Object.freeze(user);  
  
user.name = "Rahim";  
  
console.log(user.name);
```

Output:

```
Duke
```

68. Output Prediction — Example 5

```
const parent = {  
  role: "admin"  
};  
  
const user = Object.create(parent);  
  
user.name = "Duke";  
  
console.log(Object.keys(user));
```

Output:

```
["name"]
```

`role` inherited হওয়ায় `Object.keys()` -এ আসবে না।

69. Output Prediction — Example 6

```
const parent = {  
  role: "admin"  
};  
  
const user = Object.create(parent);  
  
user.name = "Duke";  
  
for (const key in user) {  
  console.log(key);  
}
```

Potential output:

```
name  
role
```

কারণ `for...in` inherited enumerable property-ও দেখতে পারে।

70. Practical Example — Read-Only Configuration

ধরো application configuration:

```
const config = {  
  apiUrl: "https://api.example.com",  
  version: "1.0.0"  
};  
  
Object.freeze(config);
```

এখন accidental mutation:

```
config.apiUrl = "wrong-url";
```

হবে না।

এটা configuration objects-এর ক্ষেত্রে useful হতে পারে।

71. Practical Example — Hidden Metadata

কখনও object-এর সাথে metadata রাখতে চাই, কিন্তু সাধারণ enumeration-এ দেখাতে চাই না।

```
const user = {  
  name: "Duke"  
};  
  
Object.defineProperty(user, "_internalId", {  
  value: 12345,  
  enumerable: false  
});
```

এখন:

```
Object.keys(user);
```

শুধু:

```
[ "name" ]
```

দেবে।

কিন্তু:

```
user._internalId
```

দিয়ে value access করা সম্ভব।

তাই আবার মনে রাখো:

Non-enumerable $\neq$ private.

72. Practical Example — Controlled Property

```
const product = {};  
  
Object.defineProperty(product, "price", {  
  value: 1000,  
  writable: false,  
  enumerable: true,  
  configurable: false  
});
```

এখন:

```
product.price = 2000;
```

কাজ করবে না।

কিন্তু:

```
Object.keys(product);
```

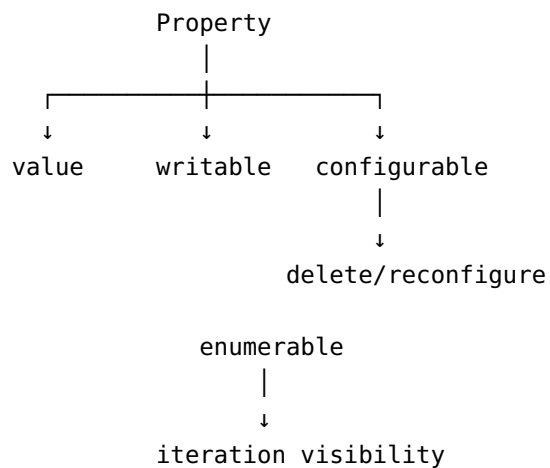
এতে:

```
["price"]
```

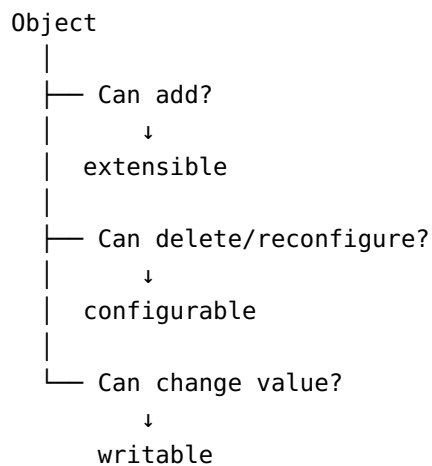
দেখাবে, কারণ `enumerable: true` ।

73. Descriptor Mental Model

একটি property-কে এভাবে visualize করো:



74. Object Control Mental Model



আর:

enumerable

↓

"সাধারণ iteration - এ আমাকে দেখাবে?"

75. Complete Property Inspection Toolkit

একটি object inspect করার জন্য:

Single property descriptor

```
Object.getOwnPropertyDescriptor(obj, key);
```

All descriptors

```
Object.getOwnPropertyDescriptors(obj);
```

Enumerable string keys

```
Object.keys(obj);
```

Enumerable string values

```
Object.values(obj);
```

Enumerable string key-value pairs

```
Object.entries(obj);
```

All own string keys

```
Object.getOwnPropertyNames(obj);
```

All own Symbol keys

```
Object.getOwnPropertySymbols(obj);
```

All own keys

```
Reflect.ownKeys(obj);
```

76. Complete Object Control Toolkit

Add restriction

```
Object.preventExtensions(obj);
```

Seal

```
Object.seal(obj);
```

Freeze

```
Object.freeze(obj);
```

Check extensibility

```
Object.isExtensible(obj);
```

Check sealed

```
Object.isSealed(obj);
```

Check frozen

```
Object.isFrozen(obj);
```

77. Big Comparison Table

Feature	Normal	Prevent Extensions	Seal	Freeze
Add property	✓	✗	✗	✗
Delete property	✓	✓*	✗	✗
Change value	✓	✓*	✓*	✗
Reconfigure	✓	✓*	✗	✗

* existing property's descriptor-এর ওপর নির্ভর করে।

78. Enumeration Comparison Table

Method	Own	Inherited	Enumerable Only	Non-enumerable	Symbol
<code>Object.keys()</code>	✓	✗	✓	✗	✗
<code>Object.values()</code>	✓	✗	✓	✗	✗
<code>Object.entries()</code>	✓	✗	✓	✗	✗
<code>for...in</code>	✓	✓	✓	✗	✗
<code>getOwnPropertyNames()</code>	✓	✗	✗	✓	✗
<code>getOwnPropertySymbols()</code>	✓	✗	✗	✓	✓
<code>Reflect.ownKeys()</code>	✓	✗	✗	✓	✓

79. Practice Tasks

Task 1 — Read-Only Property

`user` object-এ `id` property তৈরি করো।

Requirements:

```
value = 101
writable = false
```

```
enumerable = true  
configurable = false
```

তারপর test করো:

```
user.id = 202;  
delete user.id;  
Object.keys(user);
```

Task 2 — Hidden Property

একটি object-এ password property তৈরি করে যেটা:

```
enumerable: false
```

তারপর compare করো:

```
Object.keys(user);  
Object.getOwnPropertyNames(user);
```

Task 3 — Freeze Test

একটি nested object তৈরি করো:

```
const user = {  
  name: "Duke",  
  address: {  
    city: "Dhaka"  
  }  
};
```

`Object.freeze()` করার পর test করো:

```
user.name = "Rahim";  
user.address.city = "Rangpur";
```

কোনটা change হয় এবং কেন?

Task 4 — Seal Test

একটি object seal করো।

তারপর:

```
// change  
// add  
// delete
```

তিনটি operation test করো।

Task 5 — Property Iteration

একটি object তৈরি করো যাতে থাকে:

- enumerable string property
- non-enumerable string property
- Symbol property

তারপর সবগুলো দিয়ে test করো:

```
Object.keys()  
Object.getOwnPropertyNames()  
Object.getOwnPropertySymbols()  
Reflect.ownKeys()
```

প্রতিটির output predict করো।

80. Interview Questions

Property Descriptors

1. Property descriptor কী?
2. `writable` কী করে?
3. `enumerable` কী করে?
4. `configurable` কী করে?
5. `value` কী?
6. Data descriptor এবং accessor descriptor-এর difference কী?
7. `get` এবং `set` descriptor কী?

defineProperty

1. `Object.defineProperty()` কী?
2. `defineProperty()` -এর default descriptor values কী?
3. Existing property-এর descriptor কীভাবে modify করবে?
4. `writable: false` এবং `configurable: false` কি একই?
5. `enumerable: false` কি property private করে?

Object Control

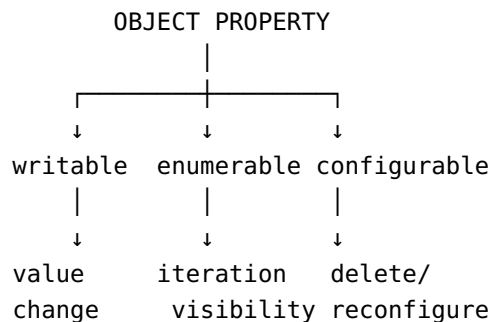
1. `Object.freeze()` কী?
2. `Object.seal()` কী?
3. `Object.preventExtensions()` কী?
4. Freeze এবং seal-এর difference কী?
5. Seal এবং preventExtensions-এর difference কী?
6. `Object.freeze()` কি deep?
7. `Object.isFrozen()` কী?
8. `Object.isSealed()` কী?
9. `Object.isExtensible()` কী?

Enumeration

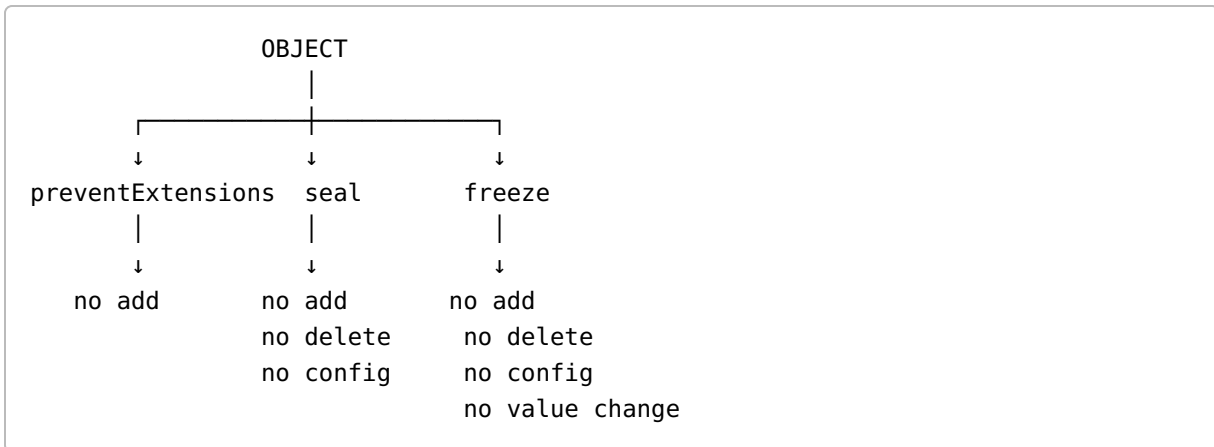
1. `Object.keys()` কী return করে?
2. `Object.entries()` কী return করে?
3. `for...in` এবং `Object.keys()` -এর difference কী?
4. `Object.getOwnPropertyNames()` কখন ব্যবহার করবে?
5. Symbol properties কীভাবে পাওয়া যায়?
6. `Reflect.ownKeys()` কী দেয়?
7. Own এবং inherited properties-এর difference কী?

81. Final Mental Model

পুরো section-টা এইভাবে মনে রাখো:



তারপর object-level control:



আর property discover করার সময়:

```
Object.keys()  
↓  
own + enumerable + string
```

```
for...in  
↓  
enumerable + string  
↓  
own + inherited
```

```
Object.getOwnPropertyNames()  
↓  
all own string keys
```

```
Object.getOwnPropertySymbols()  
↓  
all own Symbol keys
```

```
Reflect.ownKeys()  
↓  
all own string + Symbol keys
```

82. Golden Rules

Rule 1: `writable` controls whether the property's value can be changed.

Rule 2: `enumerable` controls whether the property appears in common enumeration operations.

Rule 3: `configurable` controls whether the property can be deleted or reconfigured.

Rule 4: `enumerable: false` does NOT mean private.

Rule 5: `Object.freeze()` is shallow.

Rule 6: `Object.seal()` allows existing writable values to change.

Rule 7: `Object.preventExtensions()` primarily prevents adding new properties.

Rule 8: `Object.keys()` only returns own enumerable string keys.

Rule 9: `for...in` can include inherited enumerable string keys.

Rule 10: `Reflect.ownKeys()` is the broadest standard own-key discovery tool.

83. Mastery Checklist

এই section শেষ করার পর তুমি explain করতে পারবে:

- [] Property descriptor কী
- [] `value`
- [] `writable`
- [] `enumerable`
- [] `configurable`
- [] Data descriptor
- [] Accessor descriptor
- [] `get`
- [] `set`
- [] `Object.getOwnPropertyDescriptor()`
- [] `Object.getOwnPropertyDescriptors()`
- [] `Object.defineProperty()`
- [] `Object.freeze()`
- [] `Object.seal()`
- [] `Object.preventExtensions()`
- [] `Object.isFrozen()`

- [] `Object.isSealed()`
 - [] `Object.isExtensible()`
 - [] Shallow freeze
 - [] `Object.keys()`
 - [] `Object.values()`
 - [] `Object.entries()`
 - [] `for...in`
 - [] `Object.getOwnPropertyNames()`
 - [] `Object.getOwnPropertySymbols()`
 - [] `Reflect.ownKeys()`
 - [] Own vs inherited properties
 - [] Enumerable vs non-enumerable
 - [] String keys vs Symbol keys
 - [] Freeze vs seal vs preventExtensions
-

Final One-Line Summary

Property descriptors control individual property behavior (`writable`, `enumerable`, `configurable`), while `preventExtensions()`, `seal()`, and `freeze()` control how an entire object can be modified; enumeration methods determine which own/inherited, enumerable/non-enumerable, string/Symbol properties you can discover.